UNIVERSITY OF AZAD JAMMU & KASHMIR CHELLAH CAMPUS MUZAFFERABAD



DEPARTMENT OF SOFTWARE ENGINEERING

Digital Image Processing

(Lab Task # 02)

Submitted By:

Hasnain Malik

(2024-SE-37)

Submitted To:

Respected “Engr. Fahad Nisar”

LAB 02: Fundamentals of Digital Image Processing in Python

This comprehensive laboratory report documents the practical execution of fundamental digital image processing operations implemented using Python, OpenCV, NumPy, and Matplotlib within a controlled virtual environment (.venv). The primary objective is to investigate core low-level image attributes, manipulate multi-dimensional pixel arrays, transform color spaces, execute geometric scaling, evaluate intensity distributions via histograms, and analyze quantization effects across varying bit depths.

Laboratory Objectives & Learning Outcomes

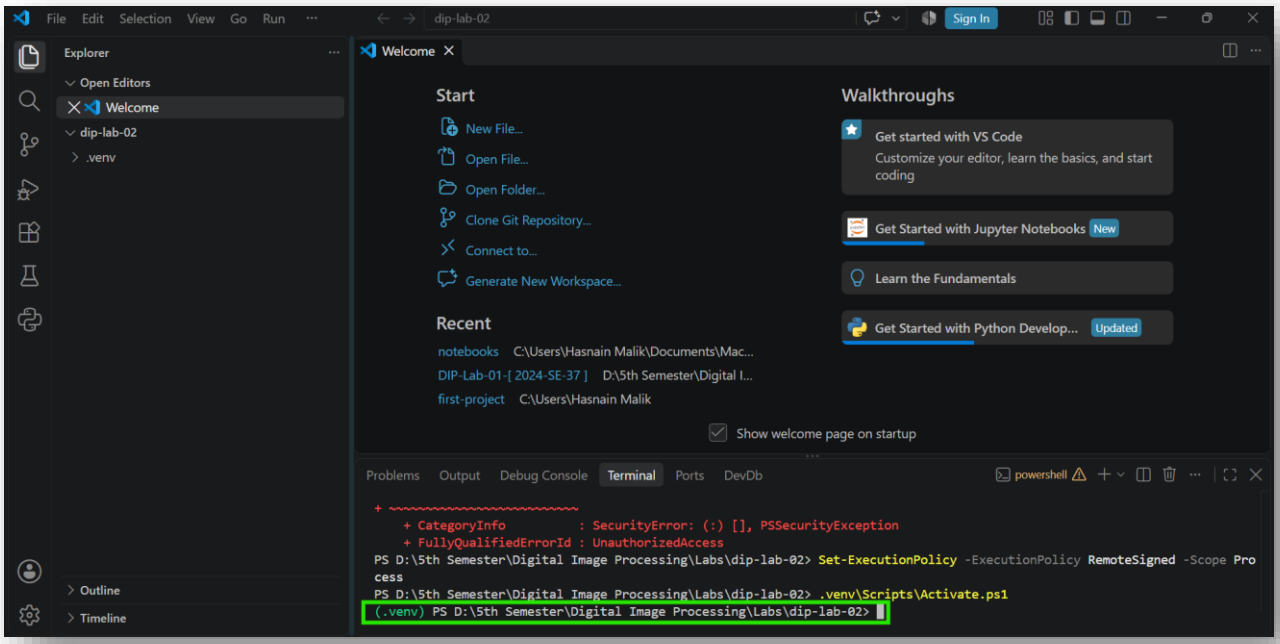
- To establish a robust Python-based computer vision development workflow utilizing virtual environments and standard industry libraries.
- To master pixel-level data inspection, multi-channel array slicing, and spatial coordinate referencing in digital images.
- To understand the mathematical and visual differences between color spaces (BGR vs. RGB vs. Grayscale) and channel separation.
- To implement precise image cropping, bilinear interpolation resizing, disk persistence, histogram analysis, and bit-depth quantization.

Step 1 & 2: Environment Configuration & Image Loading

Detailed Technical Procedure & Analysis

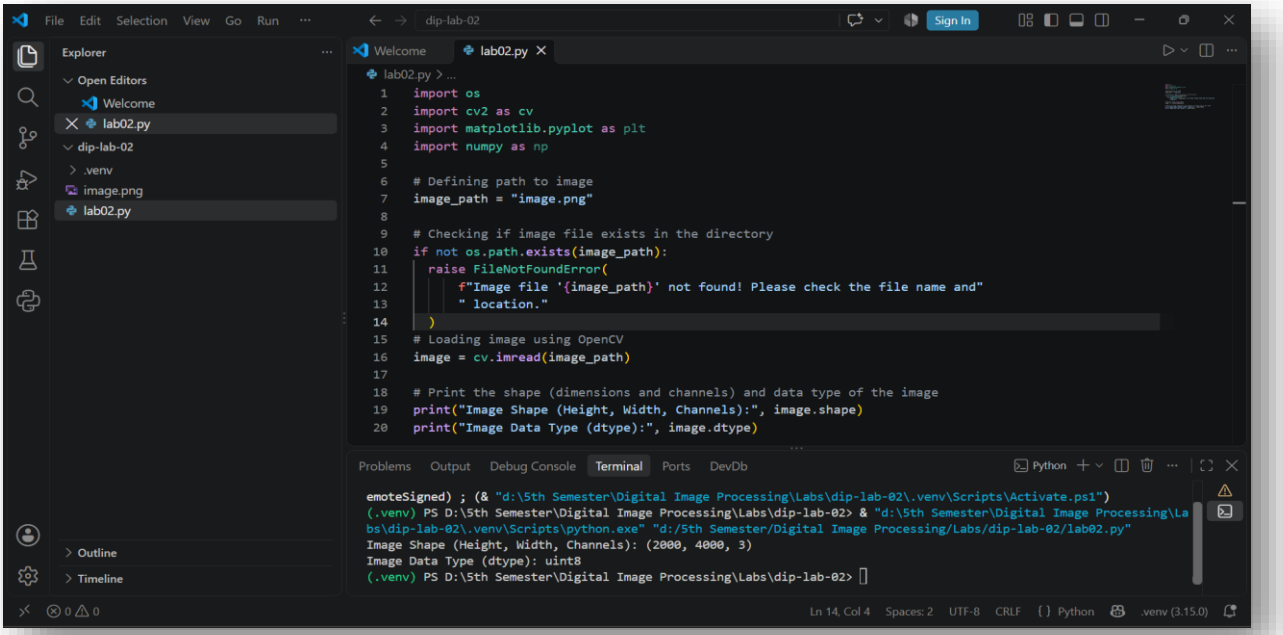
- **Virtual Environment Initialization:**
  A dedicated local virtual environment (.venv) was created and activated inside the workspace directory to isolate package dependencies and ensure academic reproducibility.
- **Library Dependencies Installation:**
  Essential computer vision and plotting packages including OpenCV (`opencv-python`), NumPy, and Matplotlib were installed and verified via terminal feedback.
- **Image Ingestion & Shape Verification:**
  The source file `image.png` was loaded into memory using `cv.imread()`. The resulting NumPy ndarray dimensions were inspected, returning a shape of `(2000, 4000, 3)` indicating a high-resolution image with 2000 rows (height), 4000 columns (width), and 3 color channels.
- **Data Type (dtype) Assessment:**
  Console verification confirmed an unsigned 8-bit integer data type (`uint8`), meaning each color intensity channel spans discrete integer values from 0 to 255.

➢ **Environment Activation:**



- **Workspace & Environment Setup:** Initialized the project directory in VS Code and successfully created and activated a dedicated Python virtual environment (.venv). This ensures that all required computer vision libraries and packages are isolated and securely managed for academic reproducibility.

➢ **Image Shape and Dtype Output:**

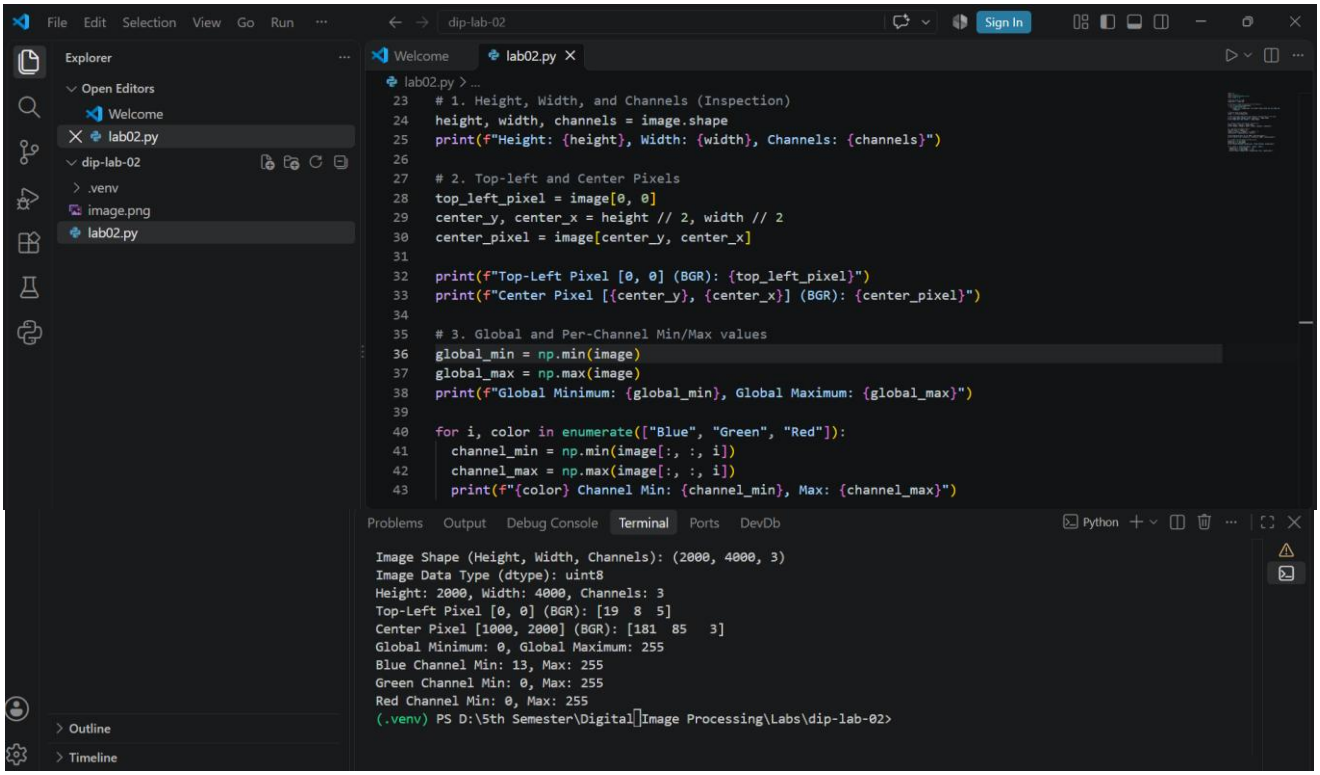


- **Image Dimensions & Shape Verification:** Inspected the multi-dimensional NumPy array dimensions after loading the source image, confirming a high-resolution frame size of 2000 rows (height), 4000 columns (width), and 3 channels (2000, 4000, 3).
- **Data Type (dtype) Assessment:** Verified the pixel data type via console output, confirming an unsigned 8-bit integer format (uint8) which restricts each color channel intensity to discrete integer values ranging from 0 to 255.

Step 3: Image Inspection & Pixel-Level Analysis

Matrix Coordinate Examination & Intensity Bounds

- **Top-Left Pixel Sampling:** Using NumPy multi-dimensional array indexing (`image[0, 0]`), the intensity values at the extreme top-left corner were extracted, yielding a BGR vector of `[19, 8, 5]`, representing near-black shadow tones.
- **Geometric Center Pixel Sampling:** Calculated the exact center coordinate `[1000, 2000]` based on height and width dimensions. Inspecting this coordinate returned a BGR vector of `[181, 85, 3]`, highlighting strong blue-dominant lighting in the center graphic.
- **Global Minimum and Maximum Extremes:** Global evaluation across the complete 3D array verified a minimum intensity of `0` (pure black) and a maximum intensity of `255` (pure saturation).
- **Per-Channel Intensity Boundaries:** Iterating across individual channels confirmed Blue channel bounds of `[13, 255]`, Green channel bounds of `[0, 255]`, and Red channel bounds of `[0, 255]`, proving dynamic tonal spread.



Step 4: Color Conversion and Channel Splitting

Color Space Transformation & Multi-Channel Plotting

- **BGR to RGB Color Space Conversion:** Since OpenCV defaults to Blue-Green-Red channel ordering while Matplotlib expects Red-Green-Blue, `cv.cvtColor()` was utilized to reorder channels for accurate visual rendering.
- **Grayscale Transformation:** Converted the 3-channel color image into a single-channel grayscale representation using weighted luminance formulas (`cv.COLOR_BGR2GRAY`).
- **Channel Deconstruction & HEX Formatting:** The image matrix was split into independent Blue, Green, and Red channels. Furthermore, the center pixel BGR values were translated into a standardized web HEX color code (`#0355b5`) for documentation.
- **Comparative Subplot Visualization:** A structured 2x3 Matplotlib subplot grid was generated to simultaneously contrast Original RGB, Grayscale, Red Channel, Green Channel, Blue Channel, and Unconverted BGR.

```python
print("\n--- Step 4: Color Conversion and Channels ---")

# 1. Convert BGR to RGB (OpenCV loads in BGR, Matplotlib expects RGB)
image_rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)

# 2. Convert to Grayscale
gray_image = cv.cvtColor(image, cv.COLOR_BGR2GRAY)

# 3. Split BGR channels
b_channel, g_channel, r_channel = cv.split(image)

# 4. Convert center pixel BGR to HEX format
# Note: OpenCV uses BGR, so we extract R, G, B accordingly for HEX
cb, cg, cr = center_pixel
center_hex = f"#{cr:02x}{cg:02x}{cb:02x}"
print(f"Center Pixel HEX Color: {center_hex}")

# 5. Plotting 2x3 subplot grid
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original RGB")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
```

```python
plt.subplot(2, 3, 3)
plt.imshow(r_channel, cmap="Reds")
plt.title("Red Channel")
plt.axis("off")

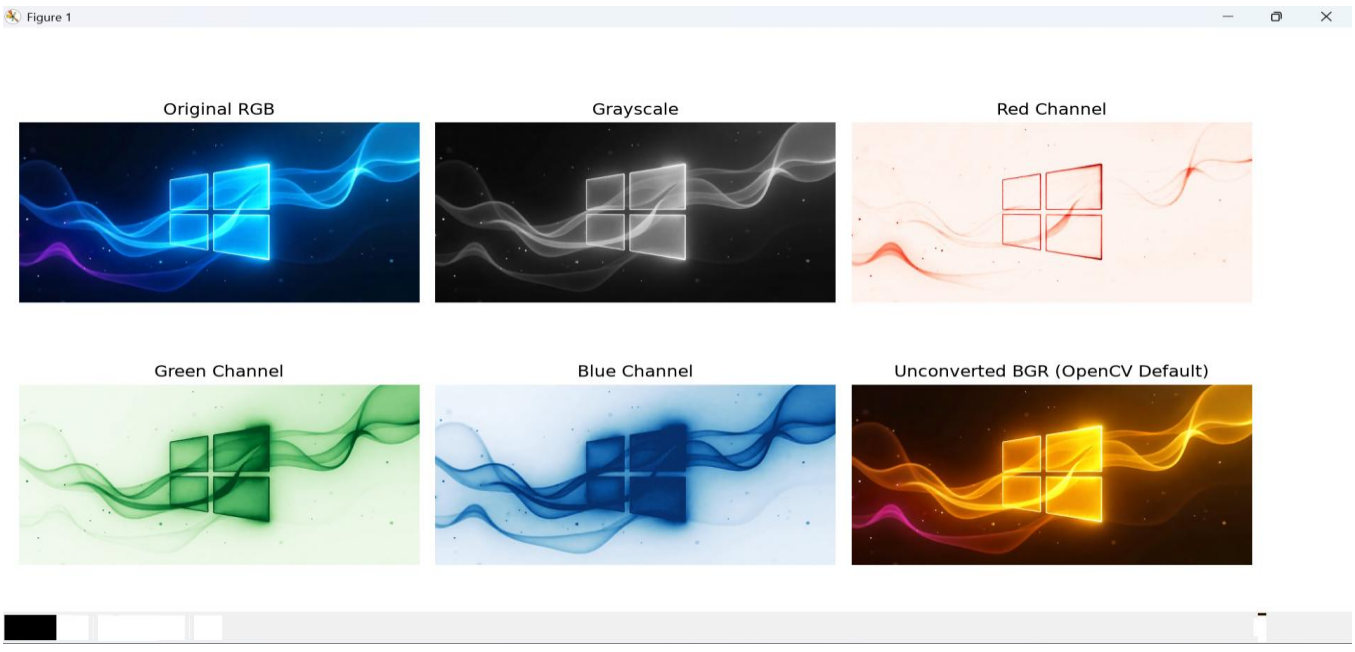
plt.subplot(2, 3, 4)
plt.imshow(g_channel, cmap="Greens")
plt.title("Green Channel")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(b_channel, cmap="Blues")
plt.title("Blue Channel")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(image)  # Unconverted BGR
plt.title("Unconverted BGR (OpenCV Default)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

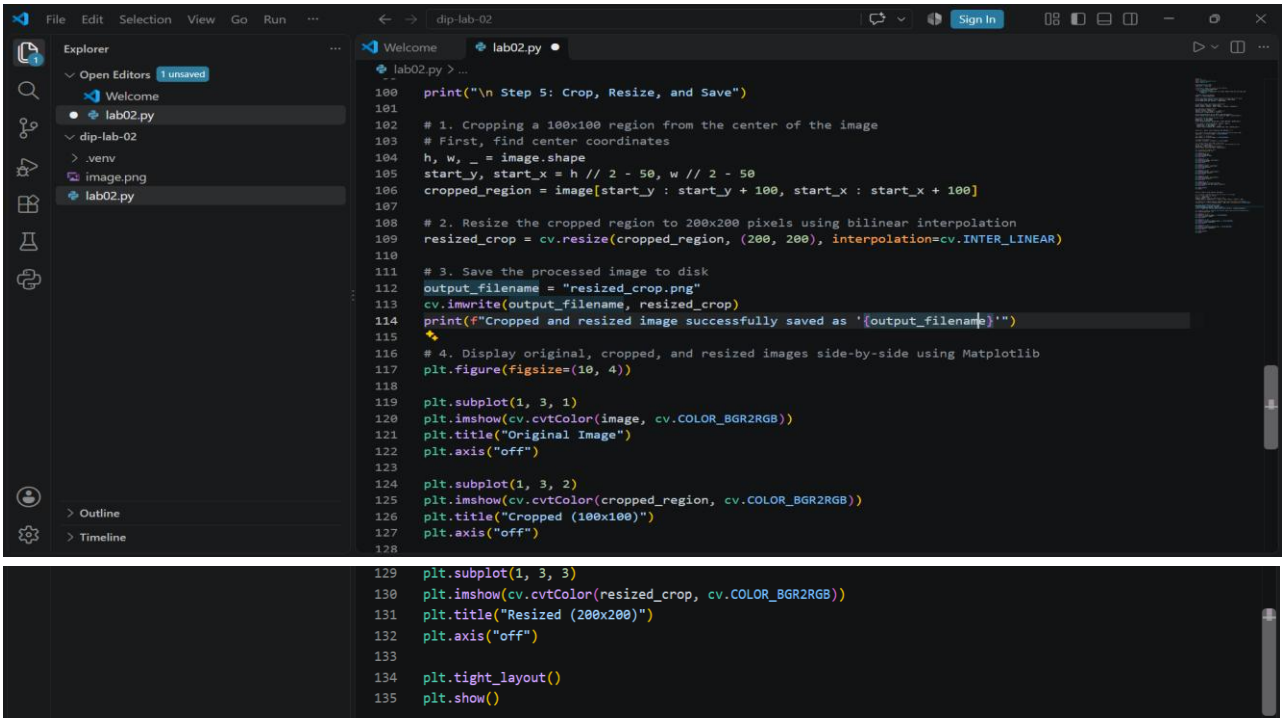
OutPut:



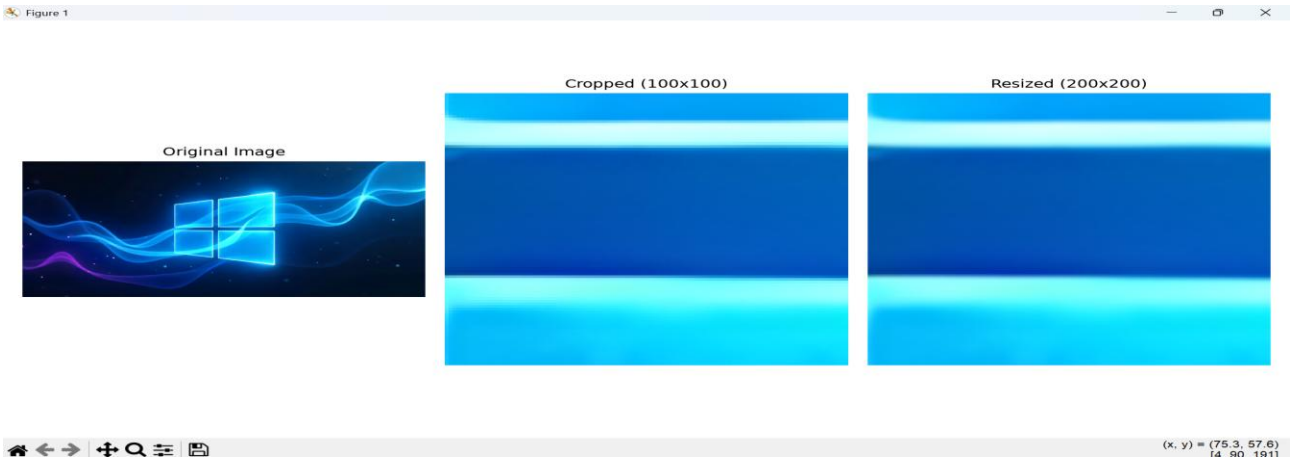
Step 5: Region Cropping, Resizing, and Saving

Spatial Slicing, Interpolation, and Disk Persistence

- **Center-Based Region Cropping:** A localized 100x100 pixel sub-region was precisely sliced from the geometric center of the image matrix using array slicing bounds derived from image dimensions.
- **Bilinear Interpolation Resizing:** The cropped patch was scaled up to 200x200 pixels using `cv.resize()` configured with bilinear interpolation (`cv.INTER_LINEAR`) to smooth pixel transitions.
- **Storage Persistence & Comparative Plotting:** The processed crop was saved to disk as `resized_crop.png`. A side-by-side comparative plot was rendered displaying Original Image, Cropped Patch, and Resized Output.



Output:

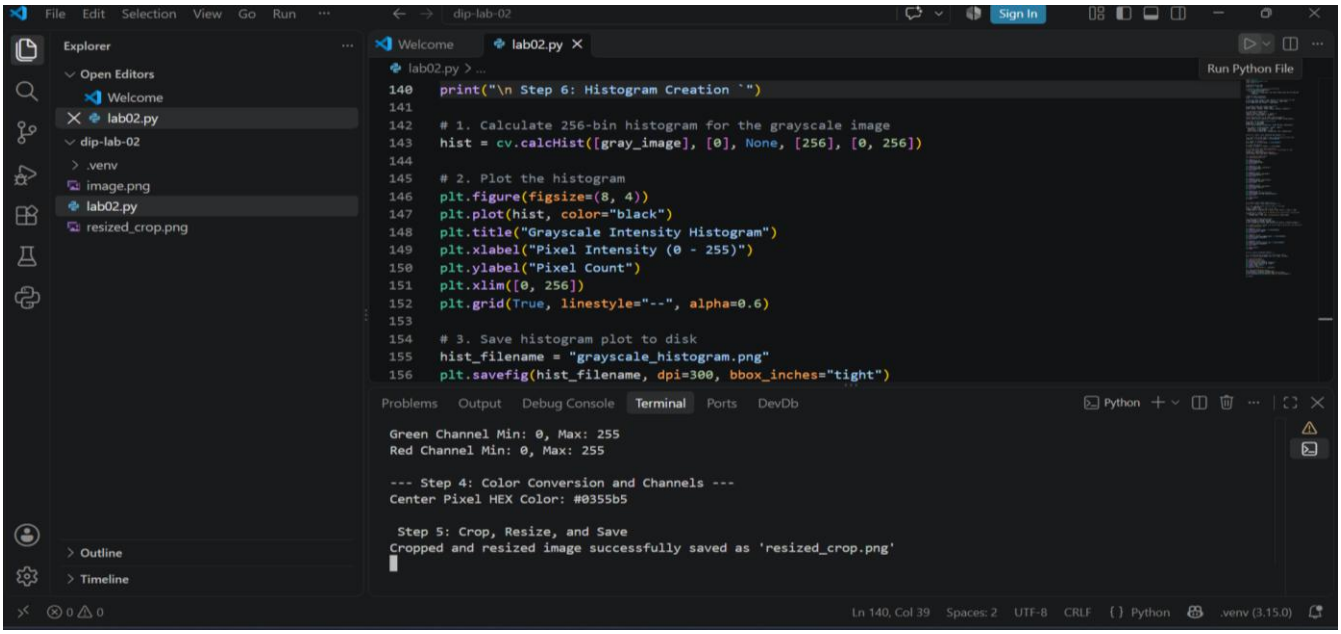


Step 6 & 7: Intensity Histogram & Bit Depth Quantization

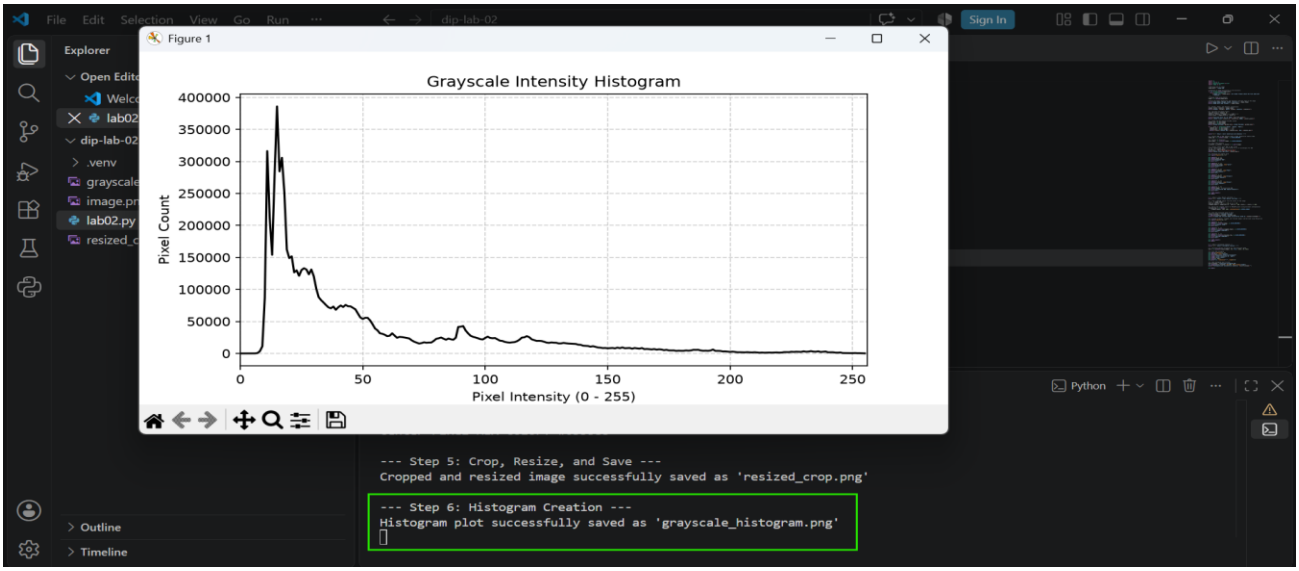
Statistical Distribution & Quantization Evaluation

- **Grayscale Histogram Computation:** Computed a 256-bin intensity histogram for the grayscale image using OpenCV's `calcHist` to map pixel frequency across intensity levels 0–255, saving the figure as `grayscale_histogram.png`.
- **Bit Depth Quantization Methodology:** Constructed an 8-bit horizontal grayscale gradient and systematically quantized it into 4-bit, 2-bit, and 1-bit intensity levels to evaluate information loss and banding.
- **Unique Intensity Verification:** Console outputs confirmed unique intensity counts of 256 (8-bit), 16 (4-bit), 4 (2-bit), and 2 (1-bit), proving exponential reduction in tonal resolution.

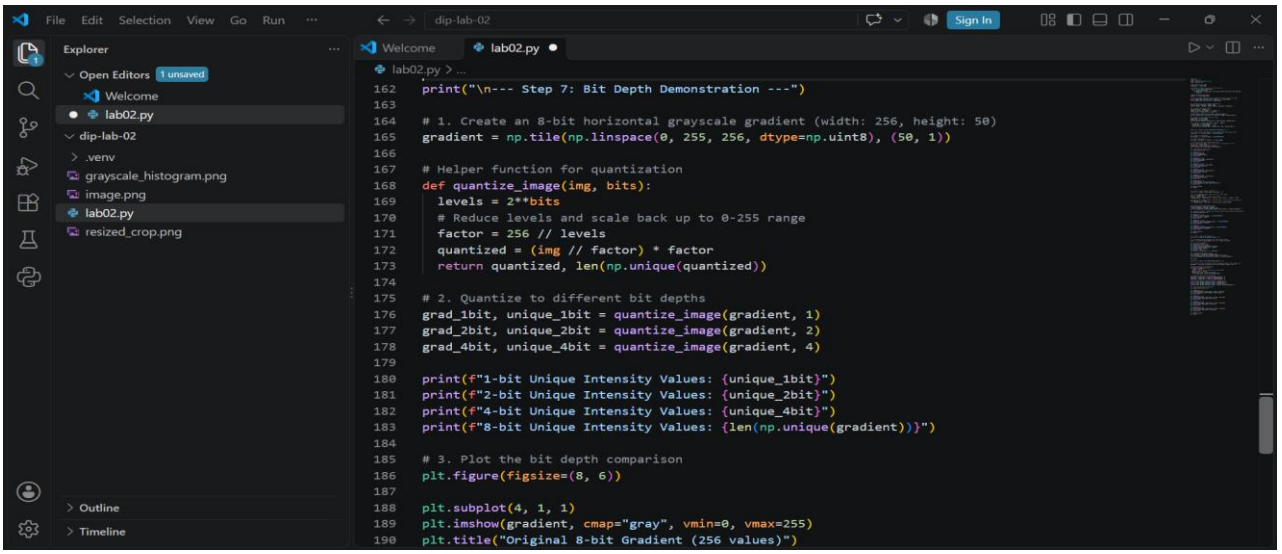
➢ **Intensity Histogram:**



Output:



➢ Bit-Depth Quantization:



```
print("\n--- Step 7: Bit Depth Demonstration ---")

# 1. Create an 8-bit horizontal grayscale gradient (width: 256, height: 50)
gradient = np.tile(np.linspace(0, 255, 256, dtype=np.uint8), (50, 1))

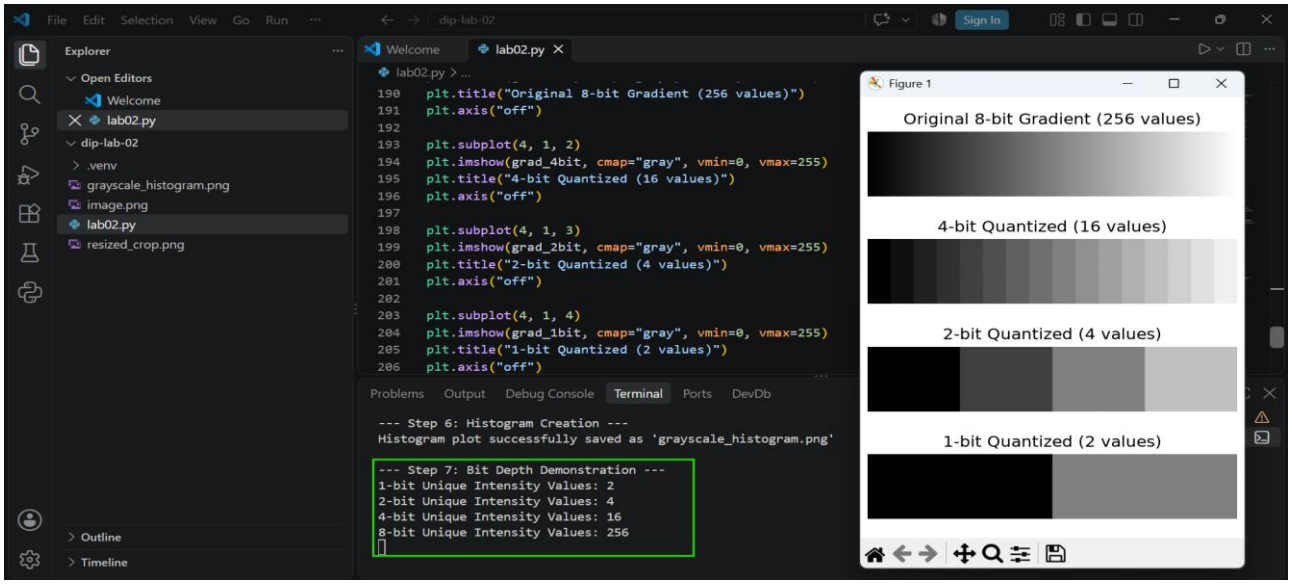
# Helper function for quantization
def quantize_image(img, bits):
  levels = 2**bits
  # Reduce levels and scale back up to 0-255 range
  factor = 256 // levels
  quantized = (img // factor) * factor
  return quantized, len(np.unique(quantized))

# 2. Quantize to different bit depths
grad_1bit, unique_1bit = quantize_image(gradient, 1)
grad_2bit, unique_2bit = quantize_image(gradient, 2)
grad_4bit, unique_4bit = quantize_image(gradient, 4)

print(f"1-bit Unique Intensity Values: {unique_1bit}")
print(f"2-bit Unique Intensity Values: {unique_2bit}")
print(f"4-bit Unique Intensity Values: {unique_4bit}")
print(f"8-bit Unique Intensity Values: {len(np.unique(gradient))}")

# 3. Plot the bit depth comparison
plt.figure(figsize=(8, 6))

plt.subplot(4, 1, 1)
plt.imshow(gradient, cmap="gray", vmin=0, vmax=255)
plt.title("Original 8-bit Gradient (256 values)")
```



➢ BGR vs. RGB

- **BGR (Blue-Green-Red):**
  The default color channel order used by OpenCV for loading and processing images, where channels are structured as Blue, Green, and Red.
- **RGB (Red-Green-Blue):**
  The standard color format expected by Matplotlib and most display tools, requiring channel reordering via conversion functions for accurate visual rendering.

➢ Resolution vs. Bit Depth

- **Resolution:**
  Refers to the total number of pixels and spatial dimensions (Height $\times$ Width) of an image, determining its overall clarity and detail.

- **Bit Depth:**
  Refers to the number of bits allocated per pixel, defining the range of available color or intensity levels (shading accuracy).

➢ What a Histogram Shows vs. Does Not Show

- **What it Shows:**
  Represents the frequency and distribution of pixel intensity levels across an image, indicating whether an overall image is dark, bright, or balanced.
- **What it Does Not Show:**
  Does not provide any information regarding the spatial arrangement, structure, or exact physical location of pixels within the image.